



NIRMA UNIVERSITY
INSTITUTE OF TECHNOLOGY
ELECTRONICS AND COMMUNICATION DEPARTMENT

Special Assignment
Course Title- communication systems
Course Code-2EC301CC23

Project Title: PC to PC wireless communication using esp32
And RF modules

Report by:-
Aarav Parikh-24BEC128
Naqeeb Syed-24BEC123

Introduction

In this project, we made a wireless communication system using ESP32 and RF modules. The main aim was to send data from one computer to another without using wires.

First, we send data from the computer to an ESP32. This ESP32 works as a transmitter and sends the data through an RF module. On the other side, another ESP32 with an RF receiver gets the data and sends it back to the computer to display on the serial monitor.

While doing this, we faced problems like noise and incorrect data because RF modules are not very accurate. To solve this, we used a simple method by adding start and end symbols in the data so that only correct messages are received.

This project helped us understand how real communication systems work and what problems can occur during wireless transmission.

Objectives

- To design a simple wireless communication system using ESP32 and RF modules.
- To transmit data from one device to another without using wires.
- To understand how serial communication works between microcontrollers.
- To study the effect of noise and errors in RF communication.
- To implement a basic method to reduce errors using start and end markers.
- To display the received data correctly on the serial monitor.

Materials Used

In this project, we used a combination of hardware and software components to build a wireless communication system. Each component plays an important role in the overall working of the system.

1. Personal Computer (PC / Laptop)

We used a computer to write, upload, and monitor the program. In some cases, two computers can be used, one for the transmitter side and one for the receiver side. However, we also tested the system using a single computer by opening two serial terminals, which acts like two different systems.

The computer is mainly used for:

- Writing code in Arduino IDE
- Sending input data through serial monitor
- Displaying the received output

This helped us simulate real communication between two devices.

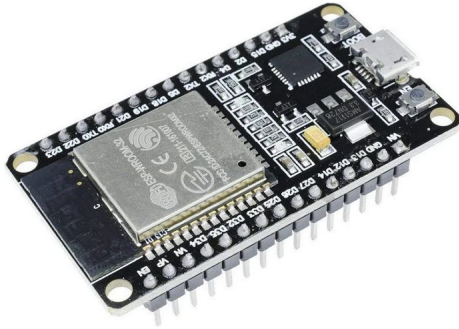
2. ESP32 Microcontroller (2 Units)

ESP32 is the main component of our project. We used two ESP32 boards:

- One as a **transmitter**
- One as a **receiver**

The ESP32 is responsible for processing data and controlling communication between devices. It has built-in UART (serial communication), which makes it easy to send and receive data.

ESP 32



We used:

- **Serial (UART0)** for communication with the PC
- **Serial2 (UART2)** for communication between ESP32 and RF modules

ESP32 is powerful and easy to program, which makes it suitable for this type of project.

3. USB Cables (Type-A to Micro USB)

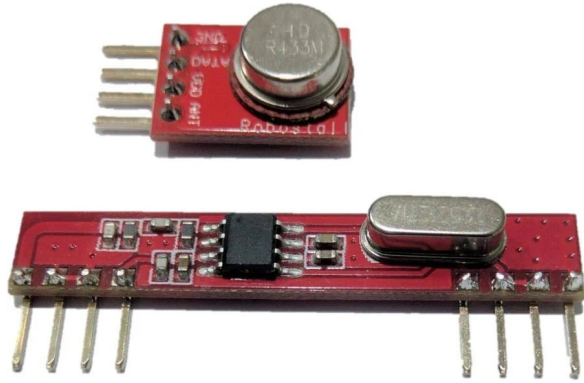
Two USB cables are used to connect both ESP32 boards to the computer. These cables serve two purposes:

- Power supply to ESP32
- Data transfer between ESP32 and PC

Without these cables, we cannot upload code or monitor output.

4. RF Modules (433 MHz Transmitter and Receiver)

RF Modules (433 MHz)



We used RF (Radio Frequency) modules for wireless communication. These modules operate at 433 MHz frequency.

- **RF Transmitter**
 - Sends data wirelessly in the form of radio signals
 - Connected to the transmitting ESP32
- **RF Receiver**
 - Receives the radio signals
 - Connected to the receiving ESP32

These modules are low-cost and simple, but they are not very accurate. They are affected by noise and interference, which creates errors in data transmission. This helped us understand real-life communication problems.

5. Arduino Uno (Power Supply)

We used an Arduino Uno to provide a stable 5V power supply to the RF modules. The RF modules do not work properly with low voltage, so external power is needed.

Instead of Arduino Uno, any regulated 5V power supply can also be used. The main purpose is to ensure stable operation of the RF modules.

6. Jumper Wires

Jumper wires are used to make connections between different components like ESP32, RF modules, and power supply.

They are useful because:

- No soldering is required
- Easy to change connections
- Helpful during testing and debugging

7. Breadboard

A breadboard is used to connect all components in a neat and organized way. It allows us to build circuits without permanent connections.

Using a breadboard helps in:

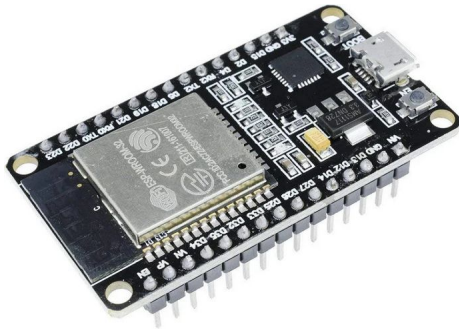
- Easy setup
- Quick changes in connections
- Avoiding wiring mistakes

Additional Notes

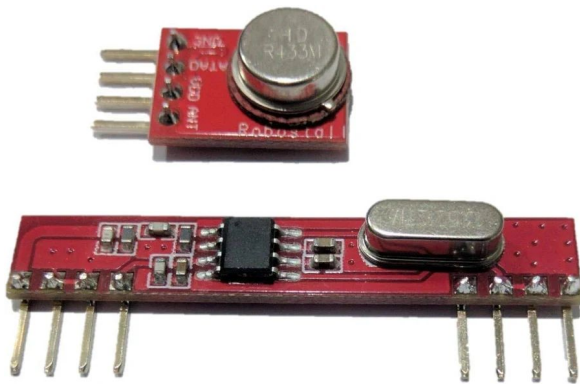
While selecting components, we focused on using low-cost and easily available parts. This makes the project simple and affordable for students.

During testing, we observed that proper wiring and stable power supply are very important for correct working. Even small connection mistakes can result in no output or incorrect data.

ESP 32



RF Modules (433 MHz)



Principle

The working principle of this project is based on wireless data communication using radio frequency (RF) signals and serial

communication (UART).

In this system, data is first sent from the computer to the ESP32 using serial communication. The ESP32 converts this data into a digital signal and sends it to the RF transmitter module through its TX (transmit) pin.

The RF transmitter converts the digital signal into radio waves and transmits it through air. These radio waves are received by the RF receiver module on the other side. The receiver converts the radio signals back into digital form.

This digital data is then given to the second ESP32 through its RX (receive) pin. The ESP32 processes the received data and sends it to the computer, where it is displayed on the serial monitor.

Since RF communication is affected by noise and interference, the received signal may contain errors. To reduce this problem, a simple framing method using start and end symbols is used so that only correct data is identified and displayed.

Thus, the system demonstrates the basic principle of wireless communication, including transmission, propagation, reception, and decoding of signals.

System Architecture

The architecture of this project consists of two main parts: the transmitter

section and the receiver section. These two sections are connected wirelessly using RF modules.

1. Transmitter Section

The transmitter side includes:

- Personal Computer (PC)
- ESP32 Microcontroller
- RF Transmitter Module

First, the user enters data through the computer using the serial monitor. This data is sent to the ESP32 through USB serial communication.

The ESP32 processes the data and sends it through its TX pin (GPIO 17) using UART communication. This data is then given to the RF transmitter module.

The RF transmitter converts the digital data into radio signals and transmits it through air.

2. Communication Channel

The communication between transmitter and receiver is done using radio frequency (RF) signals (433 MHz).

- The signal travels wirelessly through air
- It may be affected by noise and interference
- No physical connection is required

This part represents the wireless medium in the system.

3. Receiver Section

The receiver side includes:

- RF Receiver Module
- ESP32 Microcontroller
- Personal Computer (PC)

The RF receiver captures the transmitted radio signals and converts them back into digital form. This data is given to the ESP32 through its RX pin (GPIO 16).

The ESP32 reads the data using UART communication and sends it to the computer through serial communication. The received message is then displayed on the serial monitor.

4. Data Flow

The overall data flow of the system is:

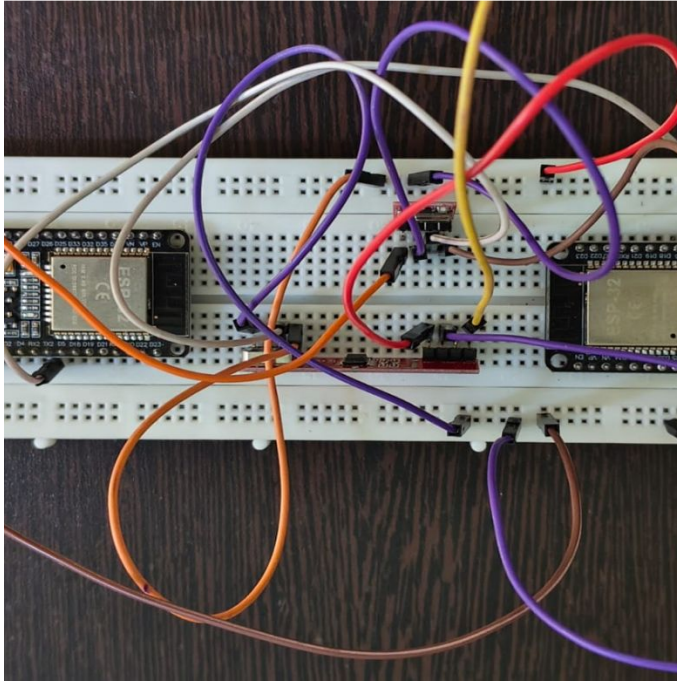
PC → ESP32 (Transmitter) → RF Transmitter → Air (Wireless) → RF Receiver → ESP32 (Receiver) → PC

Additional Explanation

The system follows a half-duplex communication method, meaning data is sent in one direction at a time. The use of UART makes communication simple between devices, while RF modules provide wireless transmission.

Due to the presence of noise in RF communication, additional techniques like framing are used to ensure correct data reception.

This architecture gives a clear idea of how data moves from sender to receiver in a wireless communication system.



Procedure

The following steps were followed to set up and connect the circuit for wireless communication using ESP32 and RF modules:

1. Initial Setup

First, all the required components like ESP32 boards, RF transmitter and receiver modules, jumper wires, breadboard, and power supply were collected. Both ESP32 boards were connected to the computer using USB cables.

Arduino IDE was used to write and upload the code into both ESP32 boards.

2. Transmitter Side Connection

On the transmitter side, one ESP32 is used along with the RF transmitter module.

- The DATA pin of RF transmitter was connected to TX2 pin (GPIO 17) of ESP32.
- The VCC pin of RF transmitter was connected to 5V supply (using Arduino Uno or external source).
- The GND pin of RF transmitter was connected to GND of ESP32.

After making the connections, the transmitter code was uploaded into the ESP32. The ESP32 reads input data from the serial monitor and sends it to the RF transmitter.

3. Receiver Side Connection

On the receiver side, another ESP32 and RF receiver module were used.

- The DATA pin of RF receiver was connected to RX2 pin (GPIO 16) of ESP32.
- The VCC pin of RF receiver was connected to 5V supply.
- The GND pin of RF receiver was connected to GND of ESP32.

After connections, the receiver code was uploaded. This ESP32 reads incoming data from the RF receiver and sends it to the computer.

4. Common Ground Connection

It was ensured that all components shared a common ground. The GND of ESP32, RF modules, and power supply were connected together to maintain proper signal reference.

5. Power Supply Setup

The RF modules were powered using a stable 5V supply. Arduino Uno was used as a power source in our setup, but any regulated 5V supply can also be used.

The ESP32 boards were powered through USB cables connected to the computer.

6. Uploading Code

- The transmitter code was uploaded to the first ESP32.
- The receiver code was uploaded to the second ESP32.

Proper COM ports were selected while uploading.

7. Testing the System

After uploading the code:

- Serial Monitor was opened for both ESP32 boards.
- Baud rate was set (e.g., 9600 or 2400 depending on setup).
- Data was typed in the transmitter side serial monitor.

The data was transmitted through RF and received on the other side.

8. Observation of Output

The received data was observed on the serial monitor of the receiver ESP32. Initially, noise and incorrect data were seen, but after applying framing technique, correct data was displayed.

9. Troubleshooting

If no output was received:

- Connections were rechecked
- Baud rate was verified
- Power supply was checked
- Wiring errors were corrected

Conclusion of Procedure

After completing all the above steps, the wireless communication system was successfully established, and data transmission was achieved between two ESP32 boards using RF modules.

Transmitter Side Code

Comments are used to explain the steps

```
void setup()

{

  Serial.begin(2400);           // PC ↔ ESP

  Serial2.begin(2400, SERIAL_8N1, 16, 17); // ESP ↔ RF (TX2)

}
```

```
void loop()

{

    // PC → ESP → RF

    if(Serial.available())

    {

        Serial2.write(Serial.read());

    }


    // Debug (optional)

    if(Serial2.available())

    {

        Serial.write(Serial2.read());

    }

}
```

Explanation of Transmitter Code

In the transmitter side, the ESP32 is used to send data from the computer to the RF transmitter module.

In the setup function, two serial communications are initialized. The `Serial.begin(2400)` is used to establish communication between the computer and ESP32. The `Serial2.begin(2400, SERIAL_8N1, 16, 17)` is used for communication between ESP32 and the RF transmitter module using TX2 and RX2 pins.

In the loop function, the ESP32 continuously checks if any data is available from the computer. If data is available, it reads the data using `Serial.read()` and sends it through `Serial2.write()` to the RF transmitter. This data is then transmitted wirelessly.

An optional debug part is also included. If any data is received back through Serial2, it is printed on the serial monitor. This helps in checking whether the transmission is working properly.

Thus, the transmitter ESP32 takes input from the computer and sends it wirelessly using the RF module.

Receiver Side Code


```

#define RX2_PIN 16
#define TX2_PIN 17

void setup()
{
  Serial.begin(2400); // PC (decoder.cpp will read this)
  Serial2.begin(2400, SERIAL_8N1, RX2_PIN, TX2_PIN);
}

void loop()
{
  while (Serial2.available())
  {
    char c = Serial2.read();

    Serial.write(c); // send to PC and show on Serial Monitor
  }
}

```

Explanation of Receiver Code

In the receiver side, the ESP32 is used to receive data from the RF receiver module and send it to the computer.

In the setup function, serial communication is initialized. The `Serial.begin(2400)` is used to communicate with the computer, where the output is displayed or read by the decoder program. The `Serial2.begin(2400, SERIAL_8N1, RX2_PIN, TX2_PIN)` is used to receive data from the RF module through RX2 (GPIO 16).

In the loop function, the ESP32 continuously checks if any data is available from Serial2. If data is available, it reads one character at a time using `Serial2.read()`.

The received data is then sent to the computer using `Serial.write()`. This allows the data to be displayed on the serial monitor and also passed to the decoder program.

Thus, the receiver ESP32 takes the wireless signal from the RF module, converts it into digital data, and sends it to the computer

Encoder (PC Side) Explanation

- We used a **C++ program (encoder.cpp)** to convert the normal message into a structured packet before sending it.
- The program uses **Windows serial communication (COM port)** to send data to ESP32.

What we used in encoder

- **C++ language**
- **Windows API (CreateFile, WriteFile)** for serial communication
- **Vector (uint8_t)** to store byte data
- **Protocol-based encoding** (custom format)

How message is sent

- User enters a message (like "HELLO")
- Each character is converted into **ASCII value**
- These values are stored as bytes

Then a **packet is created** in this format:

[Preamble][Start][Length][Data][Checksum][End]

This packet is sent through **COM port → ESP32 → RF**

Why each part is used

- ♦ **Preamble (204 repeated 4 times)**
 - Used for **synchronization**
 - Helps receiver understand that **data is starting**

- RF modules have noise → preamble stabilizes signal
- ◆ **Start Byte (170)**
 - Indicates **start of actual message**
 - Helps receiver know where real data begins
- ◆ **Length**
 - Tells how many bytes of data are coming
 - Helps receiver know **how much to read**
- ◆ **Data**
 - Actual message (ASCII values of characters)
- ◆ **Checksum**
 - Used for **error detection**
 - Calculated as:
sum of all data % 256
 - Receiver can check if data is correct or corrupted
- ◆ **End Byte (85)**
 - Indicates **end of message**
 - Helps receiver stop reading

How sending happens

- COM port (like COM3) is opened
- Baud rate set to **2400**
- Full encoded packet is sent using:

WriteFile(...)

- ESP32 receives this data through **Serial (USB)**

Why encoding is needed

- RF modules are **noisy and unreliable**
- Direct sending gives garbage
- Encoding helps:
 - Identify correct message
 - Ignore noise
 - Detect errors
 -

Encoder Explanation with Example Example Message

Let the user enter:

“HI”

ASCII values:

- H → 72
- I → 73

Step-by-step Packet Formation

- ♦ 1. Data
72 73

- ♦ 2. Length

Number of characters = 2

Length = 2

- ♦ 3. Checksum
 $72 + 73 = 145$
Checksum = $145 \% 256 = 145$

- ♦ 4. Preamble

We add 4 times 204:

204 204 204 204

- ♦ 5. Start Byte
170

- ♦ 6. End Byte
85

Final Packet

204 204 204 204 170 2 72 73 145 85

Meaning of this packet



Meaning of this packet

Part	Value	Purpose
Preamble	204 204 204 204	Signal stabilization
Start	170	Start of message
Length	2	Number of data bytes
Data	72 73	Actual message ("HI")
Checksum	145	Error detection
End	85	End of message

Code Snippets (Encoder Side)

- 1. Encoding Function

```
void encode(const string &message)
{
    data.clear();

    for(char c : message)
        data.push_back((uint8_t)c);

    calculateChecksum();
    calculateLength();
    buildpacket();
}
```

Explanation:

This function converts the input message into byte format and prepares it for

transmission by calling checksum, length, and packet building functions.

♦ 2. Checksum Calculation

```
void calculateChecksum()
{
    int sum = 0;
    for(auto c : data)
        sum += c;

    checksum = sum % 256;
}
```

Explanation:

This function calculates checksum by adding all data bytes and taking modulo 256. It helps in detecting errors at receiver side.

♦ 3. Packet Construction

```
void buildpacket()
{
    encodedMessage.clear();

    for(int i=0;i<4;i++)
        encodedMessage.push_back(PREAMBLE);

    encodedMessage.push_back(START);
    encodedMessage.push_back(checklength);

    encodedMessage.insert(encodedMessage.end(), data.begin(), data.end());

    encodedMessage.push_back(checksum);
    encodedMessage.push_back(END);
}
```

Explanation:

This function creates the final packet by adding preamble, start byte, length, data, checksum, and end byte in proper order.

♦ 4. Sending Data to ESP32


```
WriteFile(hSerial,  
    packet.data(),  
    packet.size(),  
    &bytesWritten,  
    NULL);
```

Explanation:

This line sends the encoded packet through COM port to ESP32 for wireless transmission.

Decoder Explanation (PC Side)

In the decoder part, we used a C++ program to receive the encoded packet from ESP32 through the COM port and convert it back into the original message.

The decoder reads the incoming data byte by byte and identifies the correct packet using a defined protocol.

What we used in decoder

- C++ language
- Windows API (CreateFile, ReadFile)
- Vector to store received packet
- Custom protocol decoding

How receiving works

- The decoder continuously reads data from COM port
- It does not directly trust incoming data because RF communication contains noise
- So it first tries to detect a valid packet structure

Step-by-step decoding process

♦ 1. Detect Preamble

The decoder looks for **4 continuous bytes of value 204**.

This indicates that a new packet is starting.

If preamble is not found properly, data is ignored.

♦ 2. Check Start Byte

After preamble, next byte must be:

START = 170

If this is not found:

- Packet is rejected
- Decoder resets and waits again

♦ 3. Read Length

Next byte gives the length of data:

Example:

Length = 2

This tells decoder how many data bytes to read.

- ♦ **4. Read Data**

Decoder reads exact number of bytes based on length.

Example:

72 73

These are ASCII values.

- ♦ **5. Read Checksum**

Next byte is checksum:

145

- ♦ **6. Check End Byte**

Last byte must be:

END = 85

If not:

- Packet is rejected

- ♦ **7. Extract Data**

Data is separated from full packet and stored.

- ♦ **8. Verify Length**

`if(checklength != data.size())`

If mismatch:

- Data is invalid

- ♦ **9. Verify Checksum**

$\text{sum} \% 256 == \text{checksum}$

If not equal:

- Error is printed
- Message is discarded

- ♦ **10. Decode Message**

Each byte is converted back to character:

Example:

72 → H

73 → I

- ♦ **11. Display Output**

Final message is printed:

Decoded Message: HI

Example (Full Flow)

Received packet:

204 204 204 204 170 2 72 73 145 85

Steps:

- Preamble detected ✓
- Start = 170 ✓
- Length = 2 ✓
- Data = 72 73 ✓
- Checksum = 145 ✓

- End = 85 ✓

Final output:

HI

Why decoder is important

- Filters out noise from RF
- Ensures correct message
- Detects errors using checksum
- Prevents wrong data display

Observation (Without Decoder)

When the system was tested without using the decoder program, the output observed on the serial monitor was mostly **random and unreadable characters**, as shown in the figure.

This happens because the RF receiver directly sends raw data to the ESP32, which includes:

- Noise from the environment
- Distorted signals
- Unsynchronized bits

Due to this, the serial monitor displays:

Garbage values and symbols instead of meaningful data

Reason for Garbage Output

- RF modules do not provide reliable communication
- No synchronization between transmitter and receiver
- No error detection
- Noise affects the signal

As a result, even if correct data is sent, it gets mixed with noise and becomes unreadable.

Important Observation

In some cases, correct words like **“hello”** may appear briefly, but most of the output remains incorrect. This shows that the system is transmitting data, but it is not properly decoded.

Conclusion from this Test

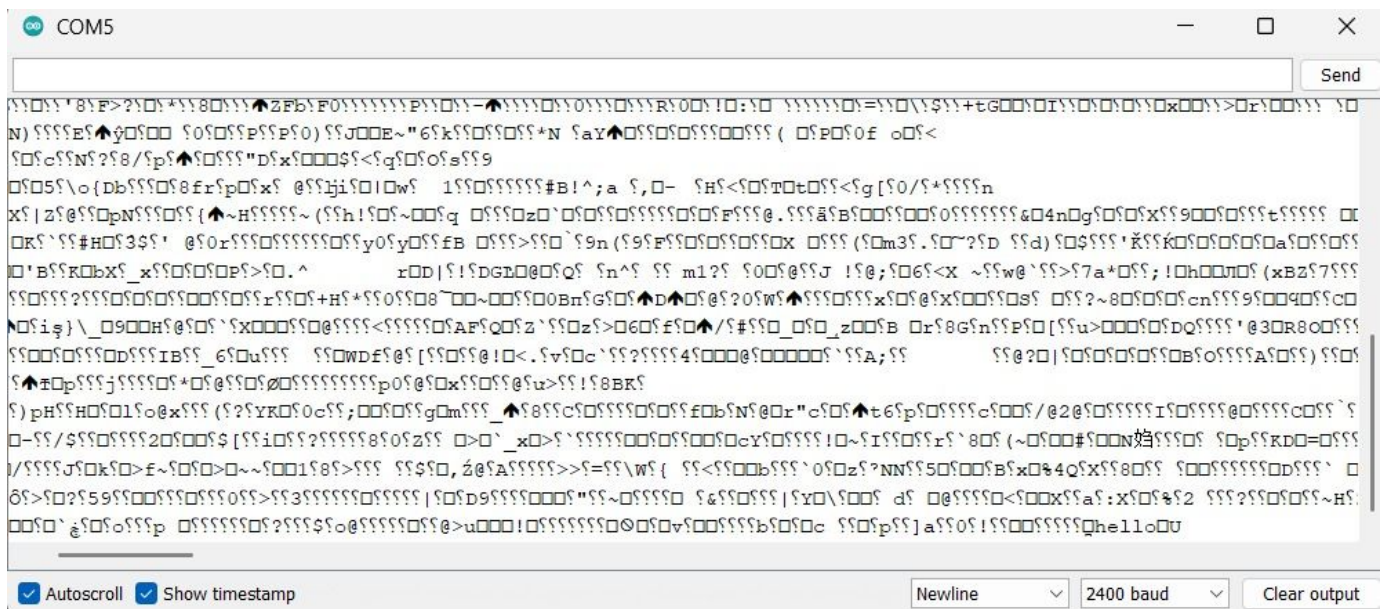
This experiment shows that:

Direct RF communication without encoding and decoding is not reliable

Why Decoder is Needed

To solve this problem, a decoder is used which:

- Detects correct packet using preamble
- Ignores noise
- Verifies data using checksum
- Displays only valid message



With Decoder

Steps to Run Encoder and Decoder

1. Compile Encoder Program

Open terminal / command prompt in folder where **encoder.cpp** is saved and run:

```
g++ encoder.cpp -o encoder
```

2. Run Encoder Program

.\encoder.exe

Then type your message when asked:

Type message: HELLO

This sends encoded data to ESP32 through COM port.

3. Compile Decoder Program

g++ decoder.cpp -o decoder

4. Run Decoder Program

.\decoder.exe

This program will:

- Continuously read from COM port
- Decode incoming packet
- Display message

Important Notes

- Make sure correct COM ports are used:
 - Encoder → COM3 (example)
 - Decoder → COM5 (example)
- Baud rate must be same:
2400 baud
- Run decoder first, then encoder (recommended)

Simple Flow

1. Run decoder
2. Run encoder
3. Enter message
4. Output appears on decoder

Conclusion

In this project, a wireless communication system was successfully implemented using ESP32 and RF modules. Data was transmitted from one system to another without using wires. Initially, direct transmission resulted in noisy and incorrect output. To overcome this, a custom encoding and decoding technique was used, which improved the reliability of communication. The use of preamble, start, length, checksum, and end markers helped in detecting and correcting errors. This project helped in understanding practical challenges in wireless communication such as noise, synchronization, and data corruption.